Kantonsschule im Lee Winterthur
Maturitätsarbeit HS 2025/26

Developing a playable simulation of markets and corporations

Written by Attila Pinter, 4a
Supervised by Patrick Hnilicka

January 5, 2026

Contents

1 Motivation

During Economy Week (Deutsch: Wirtschaftwoche), as an exercise in *business administration*, we "played" a game, where you were the board of a company manufacturing some arbitrary product and had to make decisions to increase sales, lower costs and generally beat out your competition. For me, the aspect of being able to come up with novel management ideas and compete with your friends really hooked me into the concept. This may sound hyperbolic, but during that week I genuinely woke up excited to go to school, not to actually learn stuff (and whatever) but just to play a few more rounds and (aspirationally) completely destroy my competition.

The game we got to play during Economy Week, in my opinion, had a tantalising hook,. but the interface was utterly hostile to non-directed interaction: it was pretty much an Excel table! I set off to build a better, deeper version of this simulation.

> "We do this not because it is easy, but because we thought it would be easy"
>
> **Not JFK.**

This is the running motto of this project.

Reflecting the main intention of this project, the name of the game is (currently) **Wisim** (German: **Wir**tschafts **Sim**ulation). From this point onward, all mentions of "Wisim" refer to the product of this project.

2 Concept

The goal of this project is to develop a game about leading a company in a competitive, multiplayer market. The game would have a dynamic market that has emergent properties resembling those of real economic models. Players would be able to compare their companies and see which ones are the most profitable, valuable, have the most market share, etc. and attempt to beat each other to be the most valuable company. It would be almost like "Civilization"[1] except that instead of leading a civilisation from the dawn of mankind to hundreds of years in the future, you lead a company to the next quarter.

Core Gameplay Loop

The game would have 2 repeating phases: the decisions phase and the simulation phase. During the decisions phase, each player would be able to make decisions that would lead their company to riches or to ruin. These decisions would include, but not be limited to:

- Creating various different products targeting specific sections of the market
- Controlling the promotion of said products, especially which aspects of the product they promote
- Setting up the infrastructure to produce the products
- Management of personnel

During the simulation phase there would be no input by the player, instead the game would simulate the results of the decisions made in the previous phase and compile some reports allowing the players to understand what went well or poorly such that they can improve their decisions in the next step.

[1] Civilization is a series of games about leading a civilisation from prehistoric times to the modern day. You win by getting the most victory points in science, culture, conquest or economy.

3 Design

Designing a user interface for such a game like Wisim is not the simplest of endeavours because, unlike most games, where the UI is subordinate to gameplay, in Wisim the user interface is a make-or-break factor for the game. As with many simulation games where the game acts as an interface with which the player interacts with an underlying simulation like Hearts of Iron 4[2] or OpenTTD,[3] the interface has to turn the abstract parameters and numbers of the simulation into concrete information that the player would understand in the context.

3.1 Design Philosophy and Layout

The most fundamental question in designing the game is which philosophy and layout to choose. Should I use a tabular layout and simply lay the data out with an almost 1:1 connection to the simulation, or should there be a sort of dashboard representing what an executive might get on their desk or should the game's UI visually show the physical conditions of an enterprise?

Answering this question is tough and took significant time, however there are some options that could be ruled out quickly:

- A physical representation of the enterprise that shows how machines, offices, etc would exist physically →Such a design would have a scope far outside what is reasonable within half a year.
- A simple tabular layout that resembles how the data is represented in the back-end →My judgment tells me that a design like that would make the game rather boring and unintuitive. The values displayed would be too abstracted from what they represent and would just lead to confusion.

3.1.1 Attempt #1: Tabular Dashboard

The first attempt at a design was made in April 2025 with a tabular layout with hints of a more advanced dashboard. It was very helpful for testing out various aspects of the simulation, however it proved simply too abstract to be immersive all the while many operations were painfully slow and some mechanics could not reasonably be implemented while following the existing designs (such as individual management of personnel and machines).

[2] Hearts of Iron 4 is a World War 2 simulation game, where players control nations from the 1930s to the 1950s.
[3] OpenTTD is a free transport company management game.

Concretely I didn't find it fun to play and think few other people would have found it fun.

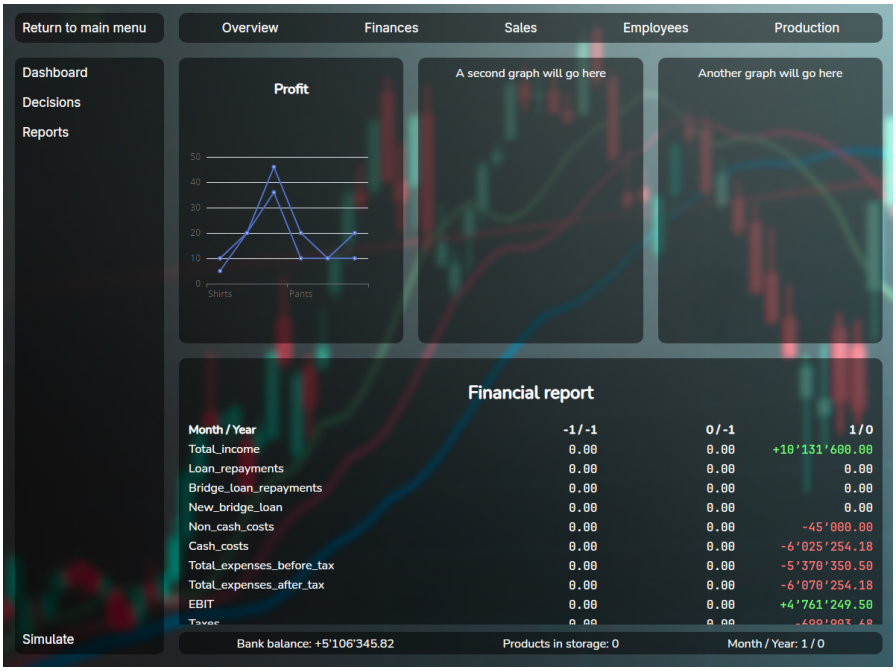

Figure 1: The first UI I made for the game

3.1.2 Attempt #2: Desktop Management system

The second attempt came a few weeks later after I saw a video on Youtube where someone was developing their own economic simulation [4] (a different aspect of the economy). They used multiple windows on an in game desktop to represent the various actions you can make. This system allows for a lot more flexibility and is more conducive to gameplay. From this point onwards I would generally follow this layout, however in different configurations.

This attempt would ultimately fail due to (self imposed) technical difficulties (further described in chapter 4 Technology).

3.1.3 Attempt #3: Desktop Management system Godot

Due to the technical difficulties I experienced in the second attempt, I tried switching to Godot, the free and open source game engine. This had similar, but different

[4] I Programmed an Economy Simulator | conaticus | https://youtu.be/BBIpAyUZq5U?si=H2j5L-wdBHYlHaUZ

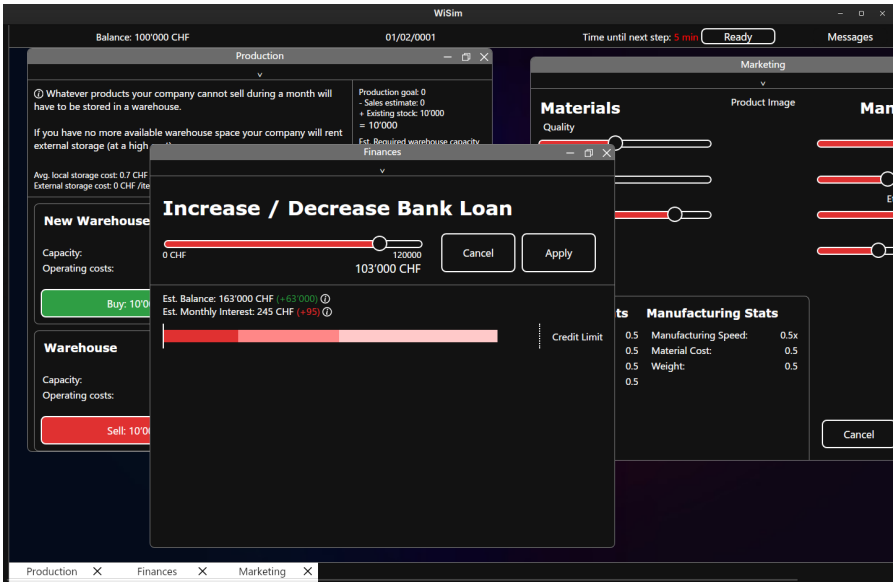

Figure 2: The second attempt at the User Interface

technical problems and came to an end not very long after it started.

3.1.4 Attempt #4: Desktop Management: Finale

Finally, I implemented a simplified version of the second attempt, this time with many technical and architectural improvements. Taking inspiration from games like Rimworld,[5] Hearts of Iron 4 and Transport Fever 2,[6] I added more graphics to the game to help the game feel more concrete and less theoretical. This version of the game, also included the product designer, which forced me to decide which product to produce (coffee machines). This version would receive the most continuous investment from me and in my opinion was the most successful of the four.

3.2 Key Design Elements

In this section I would like to showcase certain elements of the game which took more time and were more complex to develop than other parts of the game.

[5] Rimworld is a colony survival game with an AI storyteller who will trigger positive or negative events to keep you on your feet.

[6] Transport Fever 2 is a transport company management game similar to OpenTTD, but more modern.

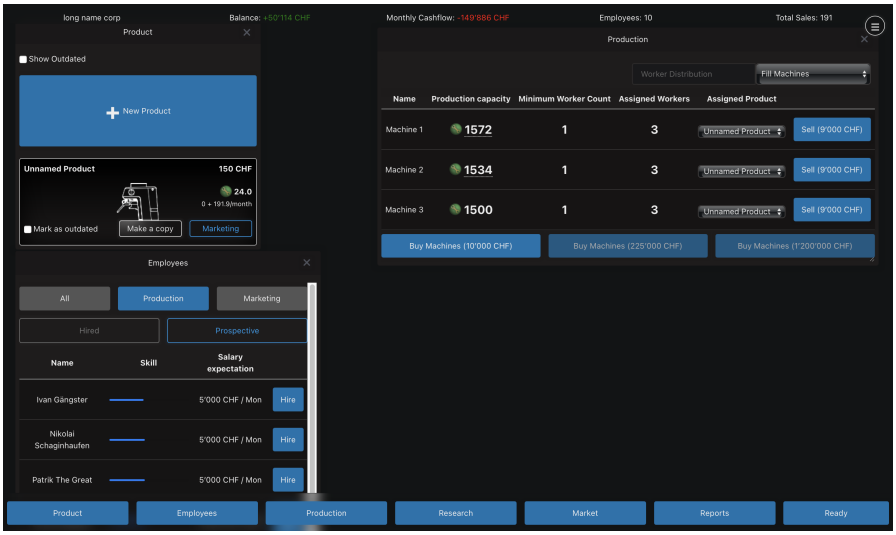

Figure 3: The final version of the user interface

3.2.1 Product Designer

The final version of the product designer was one of the most time consuming design elements of the whole game, as it not only required more complex code but also required rather deep changes to the game's internals.

In early versions of the game, the product designer was less of a product *designer* and more of a product *configurator*; the product designer was a collection of sliders, each of which would modify a single or multiple aspects of the product. This system had multiple flaws, such as:

- Being unintuitive and intimidating to new players as it just hit the player with a wall of sliders
- Being kind of boring, since you were just modifying sliders and it didn't feel like you were truly designing your own product
- Being hard to balance; balancing the effects of each slider involved complex maths with various complex curves [7] which would sometimes make obscure

[7] To give some extra context, the formula for quality of a product was:

$$2 \cdot \sqrt{\text{material quality} \cdot \text{manufacturing tech level}} + \sqrt{\text{skill of production workers}}$$

and for the production cost the formula was

$$1.1^{\text{manufacturing quality}} \cdot 0.1 + 1.1^{\text{manufacturing efficiency}} \cdot 0.1 + \frac{\text{durability}}{10}.$$

You can see how reasoning about this would be hard.

combinations overpowered in unpredictable ways.

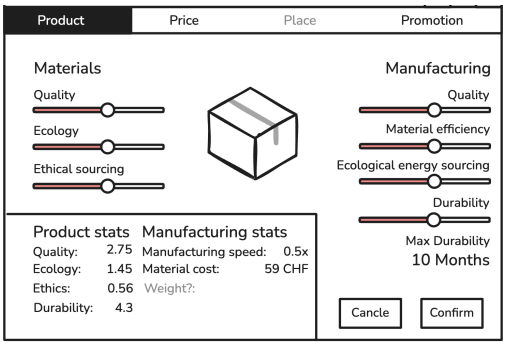

Figure 4: The original product designer

Thinking of how to fix the above problems, I remembered Hearts of Iron 4 (HOI4). HOI4 is a World War 2 simulation game, and it suffers from the same problem of making an Excel table fun to play. One of their early updates included a new tank designer in which the player chose specific components to put in their tank. This system decreases the abstractness of the content while still allowing for lots of creativity. Inspired by HOI4, I also added a similar designer system, except this time for your company's products.

Implementing the new product designer did require some major changes to the game's scope: the most important of which being that I needed to choose which product would be produced (instead of a generic product), otherwise the whole product designer wouldn't make any sense. And so after (very short) deliberation, I decided that companies would produce coffee machines, for no deeper reason than there was a coffee machine sitting right in front of me.

Figure 5: Left: tank designer from HOI4, right: new product designer from Wisim

3.2.2 Prediction mode

One of the fundamental problems I and other early play testers experienced when testing early versions of the game was that it was extremely difficult to estimate how much money they would make with a certain set of decisions. They would either just take a blind guess as to whether they would make money, or have to pull out a calculator and a notebook. The game made it very difficult to find out if your products were priced well or poorly and how decisions would impact the long term.

To solve this problem, I added "Budget Mode". When you hover your mouse over the "Ready" button, you get the option to set a sales target for each product, then you can press the "Enter Budget Mode" button. When you pressed the button the colour scheme of the game would change and in the "Reports" window and at the top of the screen you would be able to see the *predicted* results of your decisions (assuming everything went according to plan). This system is probably the most useful tool for setting your prices, because it allows the player to make sure they can make money given their prices and production.

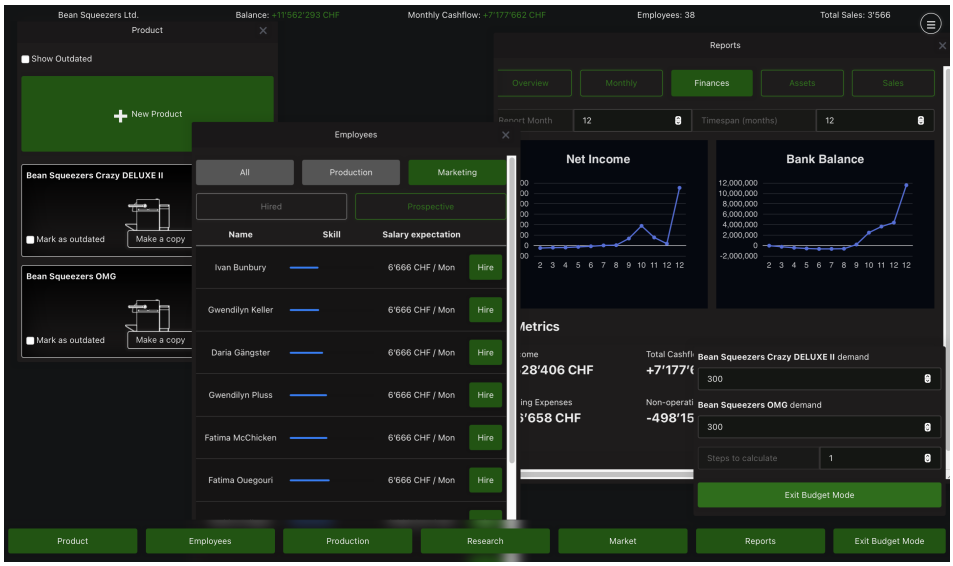

Figure 6: Prediction mode. The latest numbers shown in the graph represent the expected earnings.

4 Technology

When you start developing a piece of software of any kind, you have to pick a stack, that being: choosing which technologies you're going to build your project on top of. This choice is probably the single most important decision of any project (right after choosing what to build); getting this wrong can lead to headaches for months to come.

This project experienced two major rewrites where I swapped platform! The back-end, where the *real calculations* take place is written in "Go" (commonly referred to as Golang), for those who aren't familiar with it, it's like if Python (Tygerjython) and C [8] had a baby; it has a simple syntax like python with 90% of the performance of C. The front-end (what the user actually sees) was originally written in HTML, CSS & JavaScript using "Wails" to connect the front- and back-ends. A few months into development I was experiencing too many issues with this setup: random crashes, tight coupling [9] and unexplainable bugs. This frustrated me to such an extent that I just stopped working on the project for multiple weeks. Eventually I decided to completely rewrite the front-end in Godot, a free game engine. The problem? I didn't know Godot. The smallest of changes required hours of reading the docs and struggling around.

In the end, I came back to my senses and tried developing the front-end in JavaScript, this time learning Web-Sockets. As any developer knows, the 4th time you write something is always better than the first two times. And so I stuck with this version right until the end.

4.1 Implementation Details

The implementation of each of the game's parts strongly follows the principle of separation of concerns, driven by what I learned through the game's 4 iterations.

Over time, the game was split into 3 distinct modules:

4.1.1 The Server and Simulation

The simulation, out of all components, has had the most longevity, not having been replaced or completely rewritten a single time. The problem is that the simulation is really just a library with little ability to be played or interacted with on its own.

[8] The "C" programming language is the basis of most modern software infrastructure, being used extensively by every major operating system. It's very performant, however it has very few features, making the developer have to develop many things from scratch

[9] Coupling in software refers to how when one part of the software is changed, another related part also has to be as well.

To fix this, I added *the server*. The server (also referred to as the game server) uses the "simulation" library and acts as an interface between the front-end and the simulation: it sends data to and from the simulation to the front-end, caches user state to prevent users from having to redo all their changes if anything goes wrong between simulation steps, it sanitises input (makes sure no bad input gets into the simulation) and handles connections, saving / loading and some other functions.

4.1.2 User Interface

The problem of being able to interact with the game is not completely solved thanks to the server; unless you like typing out commands into a terminal.

Especially in the realm of video games, there are many ways to display a graphical user interface and with each solution you can *eventually* reach the same end goal. And since I had never planned to have complex graphics that would require direct access to hardware, the main factor for deciding which technology to use is how well I know it. Every technology has its quirks and its share of weirdness, however I after struggling with Raylib,[10] Dear ImGui [11] and even Godot game engine,[12] I settled for what I know, even if it isn't necessarily the "best tool for the job".

For the web interface, I chose the SvelteKit framework for JavaScript. It is a front-end web framework with a relatively simple syntax. It's very powerful and is extremely performant thanks to the fact that it compiles to regular JavaScript, HTML and CSS.

When using SvelteKit or any other web-based framework, you also need to have a web server to serve the layout to a browser, which brings us to the final component

4.1.3 The Orchestrator

The orchestrator ties everything together. It bundles everything the game needs to function into a single binary with an embedded file system. When it is run it writes the contents of its file system (the game server, web server, assets and some supporting files) into a temporary folder and runs both servers. When it's done (or one of the other servers crash), it cleans up after itself and deletes any

[10] From https://www.raylib.com/: "Raylib is a simple and easy-to-use library to enjoy videogames programming."

[11] From https://github.com/ocornut/imgui: "Dear ImGui is a bloat-free graphical user interface library for C++." Note: ImGui has bindings for other languages

[12] Godot is a free and open source game engine similar to Unity or Unreal Engine. It's an integrated environment for developing games and comes with many tools to simplify the game making process

files it created. Thanks to tools provided by Golang, this system makes it incredibly easy to compile versions for other operating systems.

4.2 Web-Sockets and Inter-process communication

One of the first non-trivial problems you experience when using multiple programming languages is how to send data from one program to the other. This is especially relevant in the context of this game because it has 2 separate processes running simultaneously (the web server and the game server). This is when Inter-process communication (IPC) comes into play and thanks to modern operating systems, there are many ways to share data between programs. Some of the most common are signals, sockets and pipes; all these systems are universally available, able to transfer data asynchronously and used in this project, however not all are suited to all tasks. To understand which method to use, we need to understand what they are.

The simplest IPC is probably the signal. Signals are sent through the operating system, interrupt the program at a certain point and run some function (by default exiting the program). What is done when a signal is received has to be defined by the recipient in advance. For example, when you close a program you send the "kill" signal, the execution of the program is interrupted and the defined action is run; this typically entails saving unsaved data, cleanly stopping the program to make sure no data is corrupted. The orchestrator uses signals to know when to close the game and clean up, however, signals aren't very useful for transferring larger amounts of data than a "kill" signal.

Far more versatile are pipes. Pipes are buffers (shared memory) that one process can write to and the other can read from. Imagine someone sharing their screen on a call, the other person can see what's happening but can't edit what's happening. So for two process to have a two-way communication channel, they need two pipes: one to the other program and another from it. The main advantage of pipes is that they are simple to implement, thanks to the fact that the operating system handles most of the hard work. Pipes are also often used by sysadmins to chain commands together: for example, on macOS or Linux. you can get a sorted list of files in a folder by entering `ls | sort`. Pipes are a really useful tool to understand, and they are used in the orchestrator to forward log messages from both servers, however they are unsuitable for communication between the front-end and the game server as they only work on the machine and cannot communicate to other machines.

The most versatile are sockets, which use can use the network to receive data using internet protocols such as TCP or UDP. Sockets are like a letterbox in a large office: each employee has an assigned letterbox and can receive letters from the internal mail service or by post. These letterboxes are called ports, and only a

single process can own a given port but can receive from many processes. Using sockets a program can also receive data from other computers on the network: for example when you open a website, your browser sends a packet (letter) to the server on port 443, along with this packet there is a return port (e.g. 31222), the server then replies with a packet containing the data that was requested sent to the port written in the request. This is the method used in Wisim. The great thing about sockets is that there is a huge amount of tools built on top of it, specifically for this project I'm mostly using Web-Sockets.

Web-Socket is a protocol built on top TCP that allows for the easy creation of a 2-way data link between the server and a browser[13]. They use a socket on both the client and server, and they are able to bypass what I like to call the "port problem": namely that the server is (usually) not allowed to query the client. [14] It does this by having the client send an HTTP request to the server, the server then "upgrades" the request to a Web-Socket and instead of replying and closing the connection, it just leaves it open. Web-Sockets are by far the easiest way to get a full-duplex (bi-direction) connection with a web-browser.

4.2.1 Implementing Web-Sockets

While Web-Sockets are very simple to implement, they have a totally different data-flow compared to HTTP: on the client you usually make an HTTP request and await a response, this is quite easy to implement, when you need data you ask for it and wait. With Web-Sockets, you have to implement asynchronous event handling because you never know when the server is going to send a new message.

On top of the previous, Web-Sockets is a binary format, that being that messages are not cleanly formatted like in HTTP with designated headers and status codes, instead they are just raw text, how the data is formatted is up to the developer. With me being *the developer*, that gives me a lot of room for stupid decisions. For example, I spent a day inventing my own data encoding protocol which mimicked a command prompt. This might not sound all that dumb until you realise that both JavaScript and my Go back-end include functions for parsing JSON[15] encoded messages.

[13] For more information on Web-Sockets see https://en.wikipedia.org/wiki/WebSocket for general information or https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API for usage details

[14] Typically, the firewall on the client device will block all incoming connections unless they are replying to a client request.

[15] JSON (short for JavaScript Object Notation) is a plain text data encoding format often used on the web. For more information see https://en.wikipedia.org/wiki/JSON

4.3 WebAssembly

Since the game has some parts written in Go and some in JavaScript, there will almost inevitably be some code that needs to be shared, lest I code 2 versions of the same function and *try very hard* to keep both versions in sync. For example, the `CalculateProductStats` function, which is used to (now this is going to surprise you) calculate the properties of products based on their components and the company. This function has to be identical on the front and back-end, otherwise players might get misled into thinking that their products are more or less good than they really are. Thankfully, other people have already had this problem and even came up with a solution. The solution, as you might guess by the section header, is called WebAssembly.

WebAssembly is a runtime for running non-JavaScript code on the browser. It is designed as a compile target for other languages so that they can run in a standardised, architecture agnostic[16] way inside a browser.[17]

Using WebAssembly along with some glue code[18], I was able to ensure that functions running on the front-end were *identical* to those running on the server. (See Figure 7)

[16] Every CPU is based on a certain Architecture, such as ARM or x86. Code built for ARM cannot run on x86 or vice versa. What WebAssembly does is that it translates WebAssembly code to compatible machine code, either just in time or ahead of time

[17] For more info on what WebAssembly is and how it works, see: https://developer.mozilla.org/en-US/docs/WebAssembly

[18] Since JavaScript is a dynamically typed language, there has to be some code that converts JS variables into statically typed Go variables

```go
func calculateProductStatsWrapped() js.Func {
    f := js.FuncOf(func(this js.Value, args []js.Value) any {
        if len(args) != 2 {
            return fmt.Sprintf("Invalid arguments: Expected 2, Got %d",
                len(args))
        }

        err := checkArguments(args)
        if err != nil {
            return err.Error()
        }

        productStats, productionLineCost := simulation.
            CalculateProductStats(product, productCompontents)

        json, err := json.Marshal(struct {
            ProductStats       simulation.ProductStats
            ProductionLineCost float64
        }{productStats, productionLineCost})

        if err != nil {
            return err.Error()
        }

        return string(json)
    })

    return f
}
```

Figure 7: Glue code for the WebAssebly version of `CalculateProductStats`

5 Simulation

When attempting to simulate a consumer market, there are quite a few approaches you could choose between. The vast majority of models fall into 2 categories:

1. Statistical models: models where the simulation uses regressions and other statistical analysis tools to determine outcomes.
2. Agent-based models: these models choose to, instead of simulating the whole market at once, instantiate individual *agents* that have the ability to independently make decisions.

There are 2 main advantages to statistical models: they are simpler to build, as you can often base them around simple regressions while still returning convincing results, and performance, unless the implementer makes grave mistakes, statistical models almost always outperform (in terms of speed) agent-based models of the same calibre.

For example, the cult classic city builder SimCity 4 was able to simulate regions with populations up to 100 million with very minor performance issues[19], while over 20 years later, games like Cities: Skylines 2[20] are still plagued by performance issues from simulating just 100'000 agents on modern, multicore hardware.

The biggest downside of statistical modelling for this project is flexibility. An agent-based model can simulate virtually anything, should the designer choose to, without necessarily major changes. This was the deciding factor for me, as the development of the game would take time, during which I would inevitably want to rework aspects of the simulation to optimise certain aspects.

5.1 How it works

The game works in 2 phases: the decisions phase and the simulation phase. During the decisions phase, players can analyse the data provided to them and adjust their companies, designing new products, hiring workers, purchasing production machines, etc. When every player is ready, the simulation phase begins, during which the decisions made in the previous phase are executed.

[19] This Kotaku article from 2014 shows off a city of over 100 million inhabitants, the Guinness world record, without any performance problems.
Link to article: kotaku.com/simcity-megacity-has-over-100-000-000-million-people-li-1629131314
World record: www.guinnessworldrecords.com/world-records/418977-largest-simcity-4-city-region

[20] Cities: Skylines 2 is a modern-day city builder released by Paradox Interactive in 2023. For more info see https://www.paradoxinteractive.com/games/cities-skylines-ii/about

5.1.1 Products

Players will develop many products during the course of the game. The products are made up of various components, these components along with the research levels of the company impact the properties of the product, for example using wooden construction will make your product slightly more ecological, whereas using plastic will make it less so.

Players have to balance quality, ecology, durability and ethics with the manufacturing difficulty, design costs and price. This forces players to attempt to target a specific customer base, lest they lose against someone who does. For example: the product that objectively has the best stats may be so expensive to produce that it has to be sold for multiple thousands of Francs, making it out of reach for the mass market.

5.1.2 Production

Along with developing a product, companies of course have to produce them. To do so they need 3 things: production machines, employees and money. Every product has a so-called "production cost", this refers to how difficult the product is to produce; a production machine will produce more products with a low production cost per month than products with a high cost, all else being equal. Every machine has a specific "production capacity". As you might expect, this refers to how much "production" can be produced per month, thus how many of a specific product.

The second ingredient of production is employees, without them machines won't run. The key gameplay impact of employees is that, as well as their salaries, the product of their skill and motivation determines how efficiently they work. If a machine has highly skilled, motivated workers, it will produce even more than its listed production capacity. As workers work on the job, they slowly increase in skill, allowing you to produce more with the same machines. This encourages players to retain their best workers and treat them well. If they are over-worked, they can burn out and become virtually useless. However, if you fire them, they might be snatched up by your competition, with them perhaps profiting off years of investment.

Last but not least is money. As with any physical good, they have to be made out of something, and that something costs money. In fact, during hard economic times the cost of materials could increase dramatically[21], which, depending on the business strategy, can impact the company's profits.

[21]This feature, along with most other external factor related features, is not implemented.

5.1.3 Marketing

For people to buy a product, they have to know it exists. With marketing, you can spread the word of your product and influence peoples' views on it at the same time. By investing large amounts in marketing quantity, you increase how many people see your product; later, when deciding which product to buy, customers only choose between the products they know. You can also influence how the product is perceived by configuring the marketing style. Marketing quantity is determined by the skill, motivation and quantity of marketing employees. Marketing style along with marketing quality will positively impact how potential customers see the promoted properties of the product.

For example, you might be able to have a lot of people buy a bad product (low quality, ecology, etc. at a high price) if your marketing is widespread and high quality enough to convince people that the product is good.

5.1.4 Consumers

The customer is simulated as a bundle of different characteristics, preferences, and needs; these all combine to produce unique behaviours for each individual agent. Taking inspiration from Economy Week, each customer has the following properties:[22]

[22] The names provided in the table may not be accurate to the corresponding implementations found in the code to simplify the explanation, as the implementation contains subtle differences for performance and technological reasons.

Property	Type	Description
Base Need	Number	The amount of products a customer wants. When their existing products wear out, they buy more.
Owned Products	List[number]	Keeps track of the remaining durability of owned products.
Known Products	List[number]	Keeps track of which products the customer knows about.
Purchasing Threshold	Number	How attractive a product has to be for the customer to buy it.
Savviness	Number	How well informed the customer is. The savvier the person, the less influenced they are by marketing.
Loyalties	List[number]	How loyal the customer is to a certain company.
Loyalty Factor	Number	How much the customer's loyalties influence their purchasing decision. [23]
Preferences	Dictionary[number]	Which aspects of a product impact the customer's decisions the most. For example, a customer could value low prices more than they do the quality of the product.

The preferences of each customer are determined when the game is created using samples of a normal distribution for each of their preferences, these are then weighted and shifted based on results of playtesting. This means that for most properties most people don't necessarily care all that much, however when looking at an individual, they might care a lot about a certain property, like getting the best value product there is.

I planned to eventually implement a system of personas as described in Malcolm Gladwell's The Tipping Point[24], such that each persona has some strong preferences others that are more muted. For example people would be some extremely knowledgeable and recommend the best products to everyone else (like influencers). This was cut due to time issues.

5.2 Simulation phase

When all companies are ready, the internal workings of the companies are simulated and marketing impressions are calculated, each customer makes a decision on which product (if any) they buy.

Customers purchase the product they perceive is the best, that is the one that

[24] The Tipping Point by Malcolm Gladwell is a non-fiction book that attempts to explain sudden, seemingly mysterious sociological changes that mark everyday life. Source: Wikipedia https://en.wikipedia.org/wiki/The_Tipping_Point

aligns the best with their preferences. This is calculated for each product using the following formula: [25]

$$\text{product opinion} = \begin{pmatrix} \text{product quality} \cdot \text{marketing style quality} \\ \dots \\ \text{product durability} \cdot \text{marketing style durability} \end{pmatrix} \tag{1}$$

$$\cdot \begin{pmatrix} \text{quality preference} \\ \dots \\ \text{durability preference} \end{pmatrix} \frac{\text{marketing quality}}{\text{customer savviness}} \tag{2}$$

$$\cdot \max\left(\frac{\text{loyalty to company}}{\text{susceptibility to loyalty}}, 1\right) \tag{3}$$

Explaining the formula: The formula calculates the dot product of the product's properties and the marketing style (putting the focus on what was mentioned in the marketing). The dot product is used here as it can be used to calculate how "aligned" two vectors are; two orthogonal vectors will have a dot product of 0 while two collinear vectors will have a dot of their products. This is all then multiplied by how affected the customer is by the marketing of the company and the person's loyalty to the company. [26] The "max" function ensures the loyalty coefficient is not less than 1.

Finally, the customer tries to buy as many of their chosen product as they need. If the product is not in stock, they don't buy anything.

5.3 An Example

There is a potential customer "John". The marketing of the products he has heard of gives John the following *perception* of the products (effects of marketing and loyalty are pre-calculated):

Product	Quality	Ecology	Ethics	Durability	Price (CHF)
Bean Squeezer	12	11	5	3	1'000
Eco Coffee 150	28	28	12	5	1'800
Coffee Supreme	55	15	4	8	20'000

John has the following preferences:

- Quality: 1.4
- Ecology: 1.8

[25] Note: There might be slight differences in the implementation. For simplicity's sake, some edge cases and optimisation are not mentioned or handled in this text.
[26] Note: Loyalty is not fully implemented in the current version of the game

- Ethics: 2.0
- Durability: 0.4
- Price: 5.5

This means that his most important criteria is that the product is well priced. John also has a purchasing threshold of 80 meaning that he will only buy products that he feels align heavily with his preferences.

When John analyses all the products he knows about, he will check how well the products align with his preferences. When he looks at "Bean Squeezer" it aligns well with his preferences:

$$\text{avrPrice} = \frac{1'000 + 1'800 + 20'000}{3} = 7600$$

$$\begin{aligned}
\text{opinion} =& 12 \cdot 1.4 && (4)\\
&+ 11 \cdot 1.8 && (5)\\
&+ 5 \cdot 2 && (6)\\
&+ 3 \cdot 0.4 && (7)\\
&+ (0.5 + (\text{avrPrice} - 1'000)/\text{avrPrice}) \cdot 3.5 && (8)\\
=& 52.58 && (9)
\end{aligned}$$

For each of the other products, he has the following opinions:

Product	Opinion	Rank
Eco Coffee 150	120.02	1
Coffee Supreme	105.97	2
Bean Squeezer	52.58	3

John likes "Eco Coffee 150" the most, and so he tries to go and buy it, upon trying to buy it he realises it's out of stock so settles for the "Coffee Supreme".

6 Tweaks and Balancing

While the core mechanics of the game might be perfectly functional, the game can only reach its full potential if there isn't just one winning strategy every time. This is where balancing comes into play. Balancing is the act of tweaking the game so that certain gameplay styles are not too powerful or too weak.

6.1 Economic Balancing

In chapter 5: Simulation, I mentioned that the preferences of people are based on samples of a normal distribution along with some weights and shifts: this is where they come into play. The key aspect that was noticed in early play testing was that price had very little impact on purchases, this had mainly to do with sims' too low "price" preference[27], so it was promptly raised. This led to another problem, namely that players couldn't produce products at that price without losing money. To fix that problem, I gave companies more money to start off with (from 200'000 CHF to 800'000 CHF) and increased the productivity of their machines.

6.2 Research and Development

Research is one of the hardest mechanics to optimise while keeping the game fun. The issue is that research having too high an effectiveness would defeat the need to ever meaningfully change anything in your company, on the other side, if research is too weak then what's the point? The solution I found to this problem was to slightly lower the effectiveness of research and only apply it to new products, this way players are forced to develop new products and maybe try out some different things compared to what they had before.

[27] This was also due to a bug I only noticed while writing this section that severely increased the population's tolerance for high prices

7 What I Learned

As with any software project, there are few things that go to plan and many lessons that were learnt along the way. One of the key lessons I learnt during the development of this project is to avoid *premature optimisation*.

7.1 Premature Optimisation

> Premature optimisation is the root of all evil (or at least most of it) in programming.
>
> **Donald Knuth**

"Premature optimisation is the act of devoting a significant amount of optimising for a task you might not truly need to optimise for."

A funny example of this was while developing budget mode. In budget mode, the player is able to see the predicted outcome of the decisions they are about to make. To be able to clearly differentiate between budget mode and normal mode, I wanted the colour scheme to change such that instead of blue being the primary colour, green was.

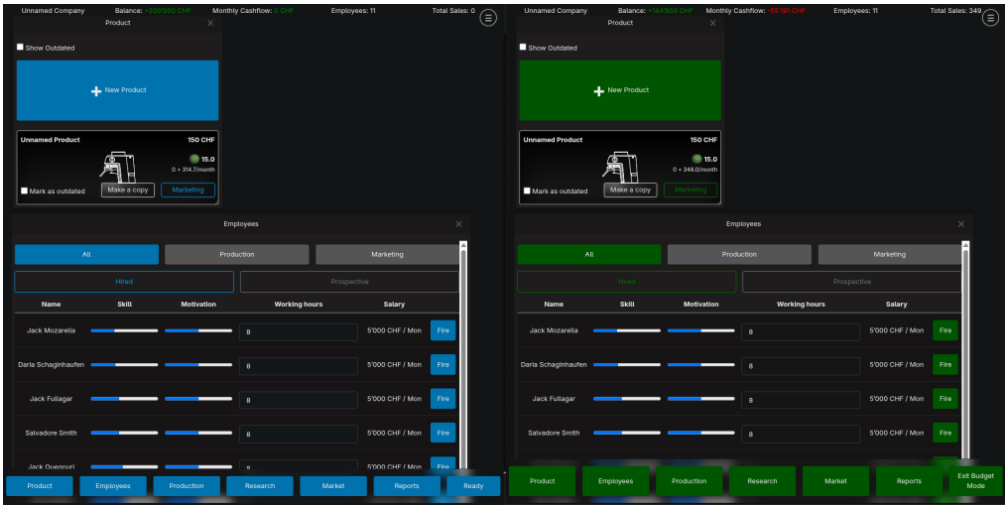

Figure 8: What I wanted to achieve

Having heard of relative colours using the CSS[28] `color()` and `color-mix` functions, I wanted to adapt my current style sheet to use these relative colours. I just had

[28] CSS is the technology used to style websites and web apps. CSS files are also referred to as style sheets.

one problem: the style sheet I was using was multiple hundreds of lines long (quite long!) and I didn't want to manually edit all of that! So what is the logical answer to this predicament? Is it to just make a new style sheet that approximated the previous one except that it had these relative colours, given that I was using a very small subset of the features of the style sheet? NO! The answer is to write a small program that reads, parses and extracts colours from a CSS file, converts the colours to relative colours (if possible) and finally rewrites the new CSS file. This, in fact, was not the most logical solution. It is not only unnecessarily trying to automate a task that realistically takes a half hour, but it also steps into the *extremely complex*[29] domain of digital colours, a domain that I had almost no knowledge in.

I ended up wasting a week of time calculating and coding this "simple" app to convert colours. The maths to derive the relative components of a colour already took multiple pages and the program itself was hundreds of lines long. This misadventure thoroughly taught me not to try to prematurely optimise something that, in the end, I didn't even need.

7.2 Testing Optimisations

While developing the economic simulation, I spent a lot of time *optimising* it, making it run quicker. Along the way, I had learned about SIMD instructions. SIMD stands for "single instruction, multiple data", this means that in the time required for one operation, the CPU can do multiple. For example, instead of doing 8 "add" operations that each take 1 cycle, it can do one "vector8 add" operation that adds two "vectors" of length 8 in a single instruction.

$$\begin{bmatrix}1\\2\\3\\4\end{bmatrix} * \begin{bmatrix}5\\6\\7\\8\end{bmatrix} = \begin{bmatrix}5\\12\\21\\32\end{bmatrix}$$

Figure 9: An example of a SIMD multiply instruction

Note: What SIMD instructions refer to as "vectors" are just arrays of a given length. The "vector multiply" operation, for example, would multiply the value at the first index of the first vector with that of the first index of the second vector and place it in the first index of the resulting vector.

Having learnt about these almost magical performance boosts, I set out to imple-

[29] For more info on digital colours, see: "Everything About Color in 25 Minutes" by Juxtopposed. https://www.youtube.com/watch?v=srRI7yMjGz0&t=6s

ment this in my game. I spent hours completely changing the order in which the simulation would calculate purchasing decisions, and making sure the data was in the right structure to work with SIMD.

All that work for **worse performance**. That's right, worse performance. The problem is that converting the data structures I was using to compatible ones for SIMD took longer than the actual calculations.

Sometimes, when programming, optimisations that you expect to increase the speed of your code dramatically perform just as well as the naive implementation, and sometimes worse. You often cannot rely on your intuition in matters of performance, especially when dealing with modern CPUs that have built-in optimisations that you would need a PhD to understand. You should always test optimisations to see if they really work.

7.3 Feature Creep

Whenever you start a big project, especially in programming, you have to fight a constant battle against what is called "feature creep". Feature creep is the urge to constantly increase the scope of a project. For example, in this project, while reworking the product designer, I thought about a new, more advanced production line system that would completely change the way the game worked. It would involve creating entire production lines, sourcing materials, buying machines that could handle specific parts of the manufacturing process, slowly specialising as you grow, etc. This, to be honest, would have been pretty cool, however it would probably have extended the development cycle of the game by a factor of two.

When making a large project, you always have to fight this urge of adding "just one more feature".

8 Conclusion and Future

This project has taught me a lot, from game design to advanced optimisations to time management, there are few things that I *didn't* learn from this project.

I am quite happy with the result of Wisim, however I can't call it a finished product yet. The game is in a playable state, but I don't think it's ready for a release yet. I plan to continue working on the game until it is truly ready, I then plan to release it on Steam.

9 Acknowledgements

Many thanks to:

My Brother Félix Pinter, my Father Frank Pinter and Felix Angerer
for proofreading and playtesting,

Elias Brugger and Andrin Ganster
for playtesting,

Patrick Hnilicka
for playtesting and supervising and

You
for reading.

10 Appendix

10.1 Download the Game

If you want to try the game out for yourself, you can download the game files by going to the URL below, clicking on "Releases" on the right, then downloading the correct binary for your device.

https://github.com/AttilaART/Wisim

For windows download the ".exe" file, for newer Macs download the "darwin-arm64" version, for older Macs download the "darwin-amd64" version and for Linux download the "linux" version.

MacOS and Linux users may have to make the file executable by running the command displayed below (replace "wisim-linux-amd64" with the name of the file you downloaded):

```
attila@archlinux ~$ cd Downloads
attila@archlinux ~/Downloads$ chmod +x wisim-linux-amd64
attila@archlinux ~/Downloads$
```

After starting the program, a browser window should open, which shows the game's main menu. If this doesn't happen, navigate to http://localhost:3000

10.2 Images

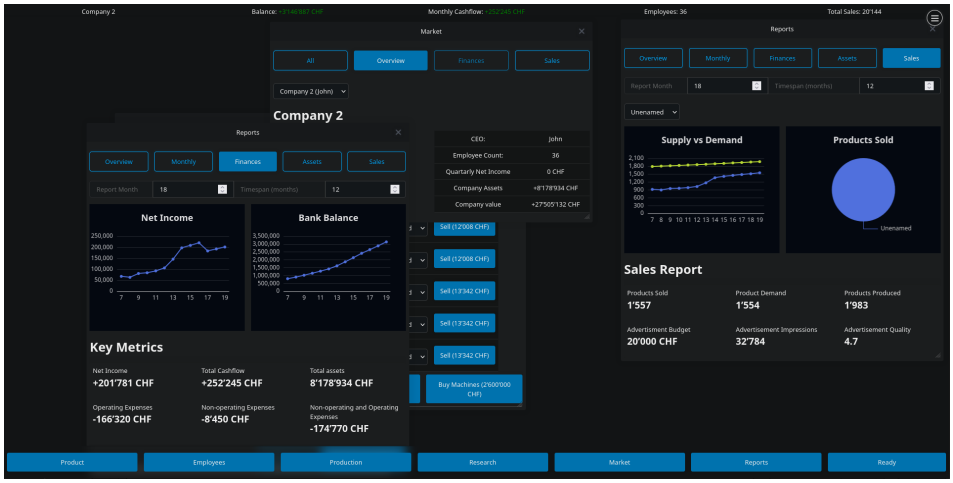

Figure 10: Screenshot of game

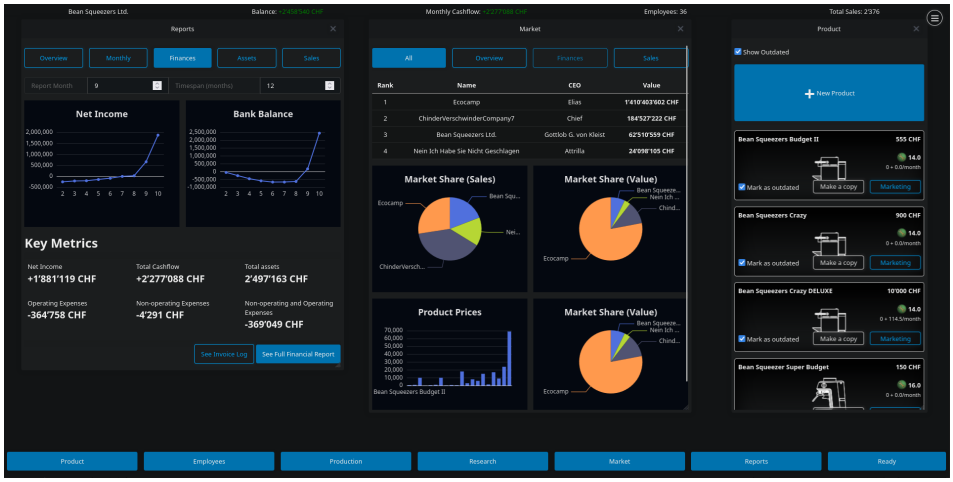

Figure 11: Screenshot of game

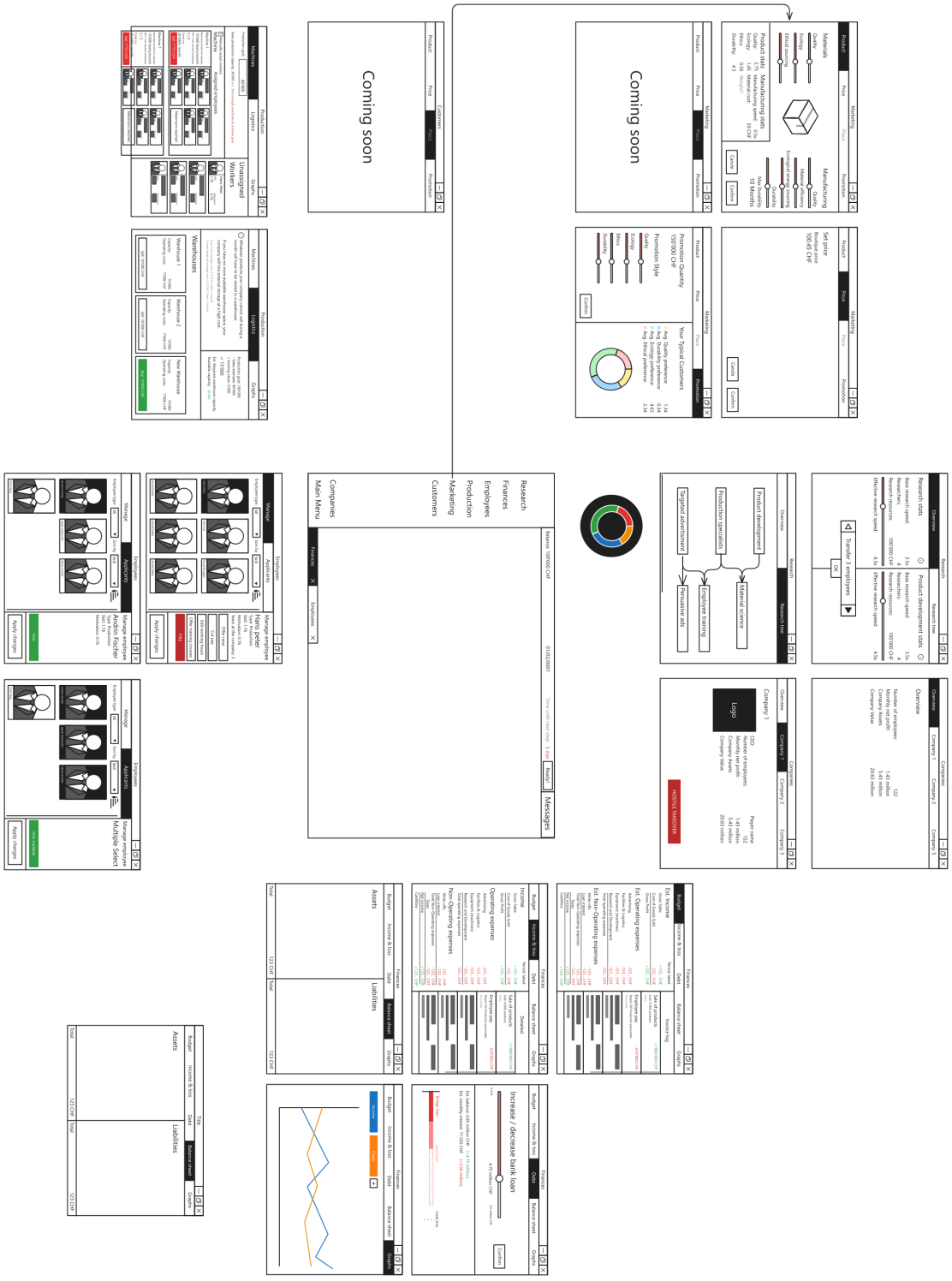

Figure 12: Design canvas of game (prototype)

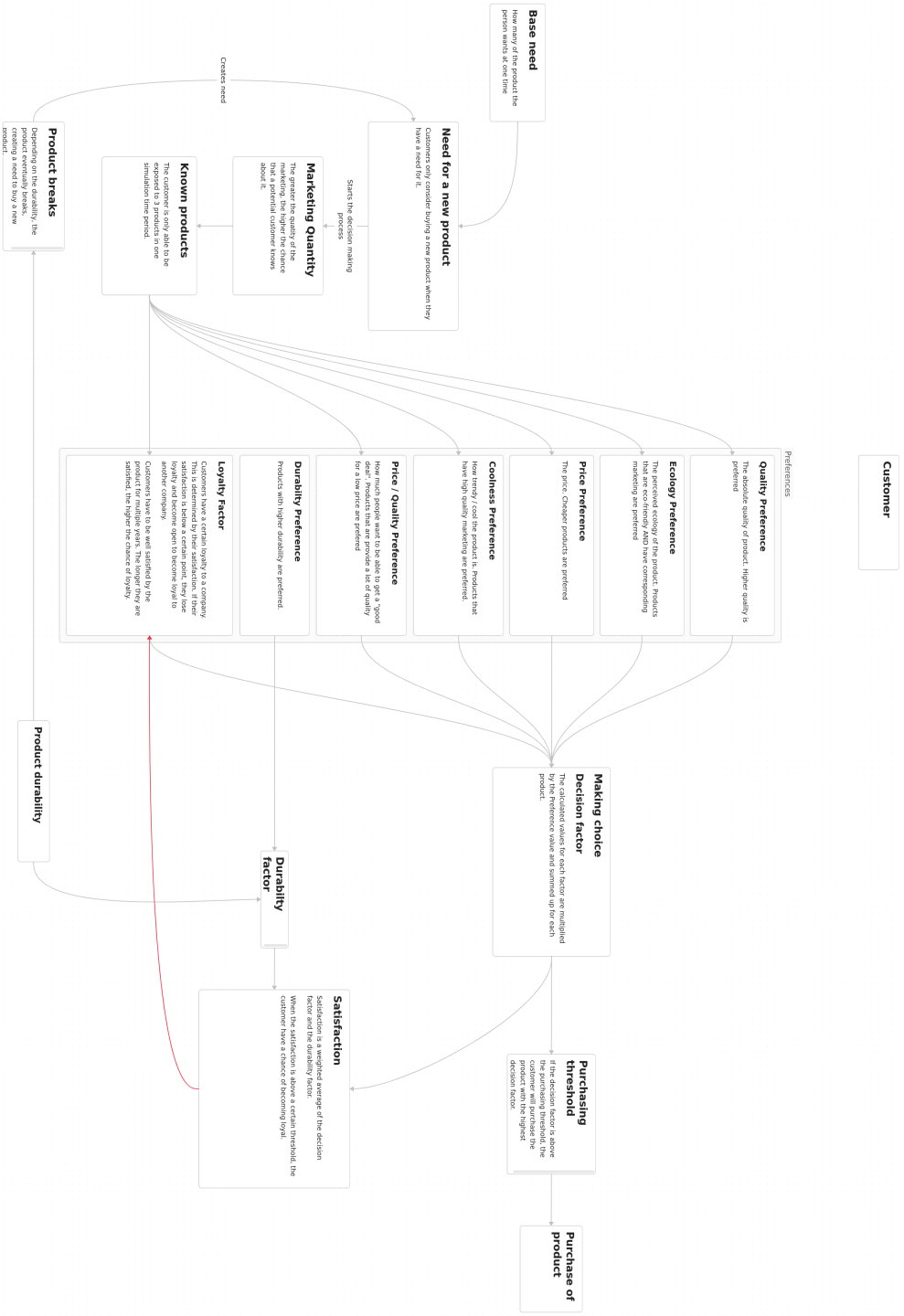

Figure 13: Decision making process of sims (prototype)